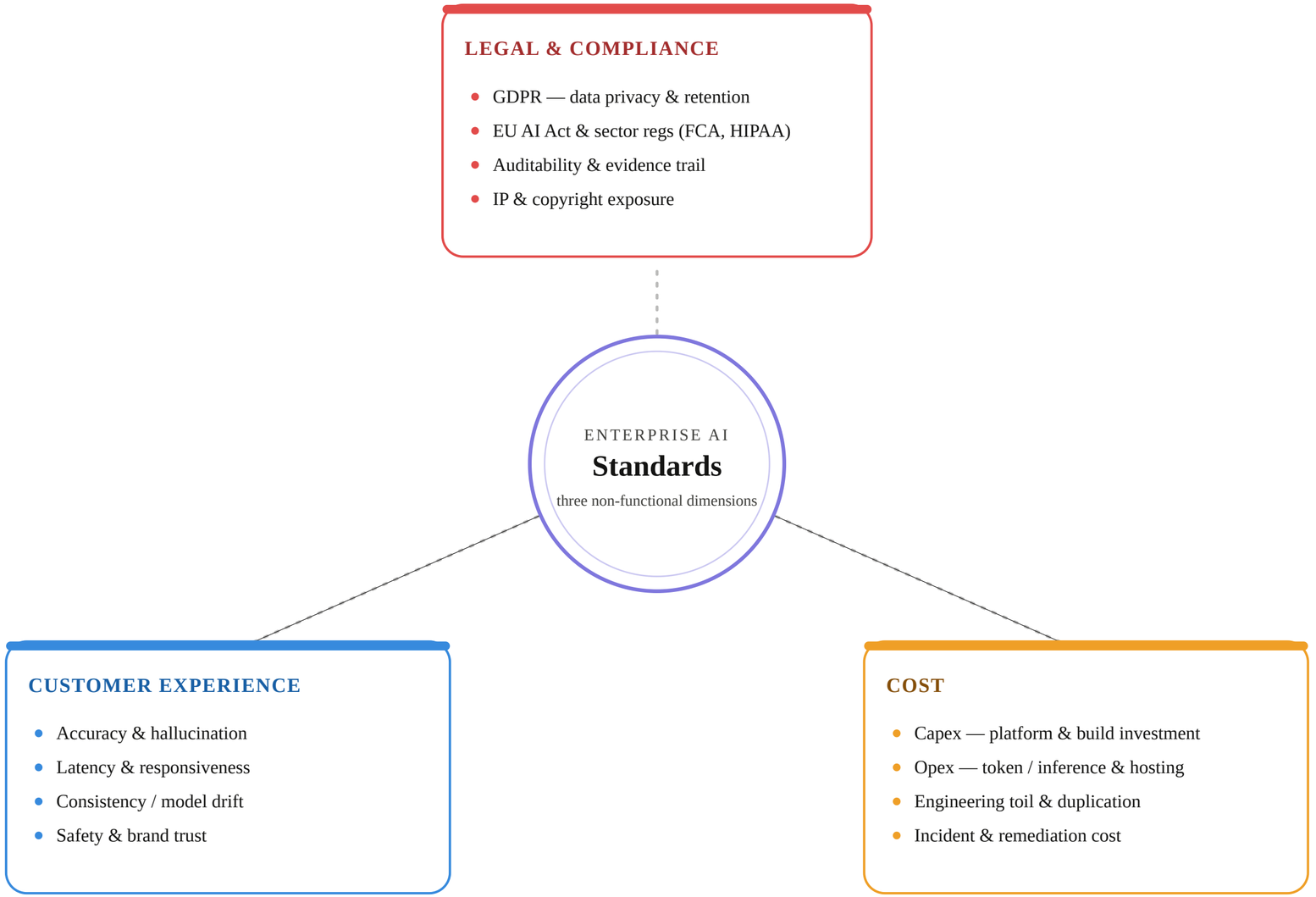
PART 2

BUILDING THE CATALOG

Enterprise AI — Three Categories of Standards

Every standard serves exactly one of three non-functional dimensions



Same Catalogue, Two Sourcing Approaches

Only one of the three categories comes with an external source of truth

COST & CUSTOMER EXPERIENCE

ORG-SPECIFIC

You set the bar

- No external authority defines "good"
- Set by your traffic, margins, model mix, tone
- Reckless for one workload, trivial for another

APPROACH · AUTHORED

- Built from first principles + heuristics
- Lens shapes the build, not where your line sits

Built, not adopted.

LEGAL & COMPLIANCE

EXTERNALLY SET

The bar is set for you

- You don't get to invent the baseline
- Inherited from what creates liability
- EU AI Act, GDPR, FCA, sector regs

APPROACH · DERIVED

- Trace each obligation to a testable standard
- Governance owns "what", EA owns "how"

Derived, not authored.

Cost & CX you iterate on — you own the bar.

Legal you inherit — non-negotiable, and where catalogue-building begins.

Building Cost & CX Standards — Three Ways

No external baseline hands you these — you source them yourself

1

FIRST PRINCIPLES

Set your own bar

- Start from margins, latency, brand voice, risk appetite
- Reason forward from "what is good for us"
- You write the threshold

TOP-DOWN · AUTHORED

2

BEST-PRACTICE FRAMEWORKS

Adopt the pattern

- AWS GenAI Lens gives the proven shape of a control
- What to measure, what knobs exist
- Adopt the pattern, then set your number

ENGINEERING SCAFFOLD

3

PRODUCTION FAILURES

Learn from failures

- Mine incidents, postmortems, near-misses
- Each failure = a candidate standard
- Evidence-driven, hard to argue with

BOTTOM-UP · EVIDENCE

First principles is top-down, failures bottom-up — the Lens is the scaffold in between.

AWS GenAI Lens — Structure for Cost & CX, You Set the Line

For Cost and CX, adopt the checklist; author the threshold yourself

COST

ONE PILLAR

Dedicated Cost Optimization pillar → ready-made slots

- Select cost-optimized models
- Balance cost vs. inference performance
- Engineer prompts for cost (length / size)
- Optimize vector stores & agent workflows

YOU SET THE LINE

- e.g. ≤ 2k tokens per call
- e.g. premium tier must be justified

Adopt the checklist · author the gate

CUSTOMER EXPERIENCE

ASSEMBLED

No single pillar → gather structure from four

- Performance Efficiency → latency
- Operational Excellence → output quality, drift
- Reliability → consistency, degradation
- Responsible AI → hallucination, safety, trust

YOU SET THE LINE

- e.g. p95 latency < 3s
- e.g. hallucination rate < 1%

Adopt the checklist · author the gate

The Lens gives you the shape and the checklist — never your org-specific number.

Structure is inherited; the threshold is authored. Adopt the pattern, then set the gate.

Legal & Compliance — You Inherit Principles, Not Standards

Risk teams and counsel give you the "what" — converting it is EA's job

THE HANDOFF

INPUT

What Risk & Legal give you

- High-level principles & obligations
- You accept them as the obligation set
- Don't re-litigate whether they apply

THE OWNERSHIP LINE

- Governance owns "what must be met"
- EA owns "how it's built & verified"

Their output is your input.

THE FILTER

KEEP / DROP

Not every principle becomes a standard

- Drop org-level → QMS, docs, registration
- Keep system-level → what a model implements
- Keep only GenAI-distinctive controls

WHERE THE REST GOES

- Org-level obligations → ISO 42001
- Generic infosec → already covered

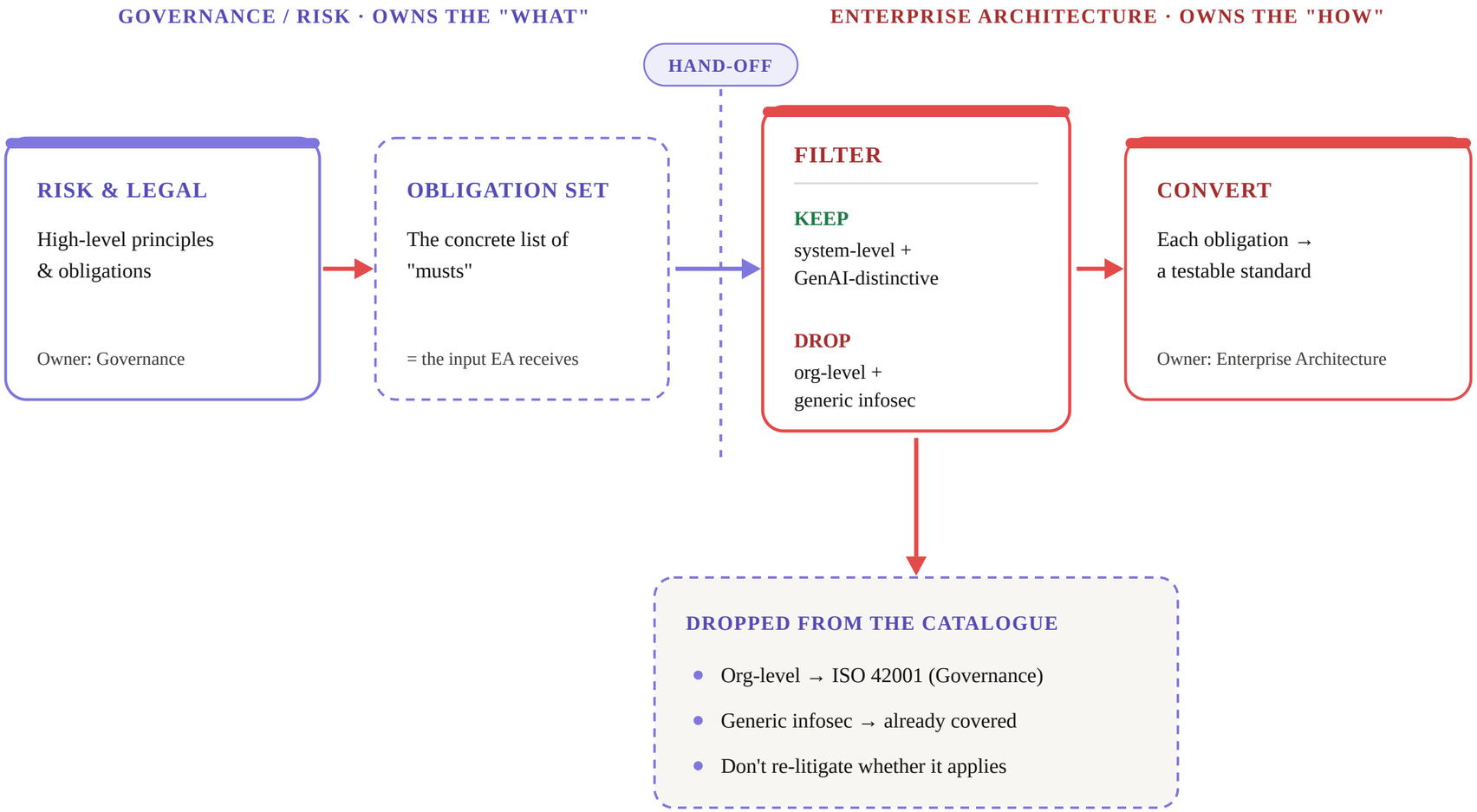
Filter before you build.

You inherit principles, not standards.

Accept the obligation set, keep the system-level GenAI-distinctive ones — then convert.

You Inherit Principles, Not Standards

The obligation set crosses from Governance to EA — EA filters, then converts



You inherit principles, not standards.

Accept the obligation set, keep the system-level GenAI-distinctive few — then convert.

Worked Example — EU AI Act: Keep vs Drop

Filtering high-risk obligations into catalogue standards

KEEP → BECOMES A STANDARD

System-level & GenAI-distinctive

Art 12 · Record-keeping
→ Trace every prompt, retrieval & output

Art 13 · Transparency
→ Disclose AI use; outputs cite their sources

Art 14 · Human oversight
→ High-risk actions need human sign-off

Art 15 · Accuracy & robustness
→ Grounding gate: answers backed by context

Each earns a gated standard in the catalogue.

DROP → NOT A STANDARD

Org-level, or already covered

Art 11 · Technical documentation
→ ISO 42001 (Governance owns it)

Art 17 · Quality management system
→ ISO 42001 / Governance process

Art 43 & 49 · Conformity + registration
→ Governance process, not code

Art 15 · Encryption, access control
→ Generic infosec, already covered

Each belongs to Governance, not EA.

One article can split in two.

Art 15 → **keep** the grounding gate (GenAI-distinctive) · **drop** the generic cybersecurity (already covered)

Filter by the control, not the article: keep what a model implements and is unique to GenAI.

Article references: EU AI Act, Chapter III (high-risk AI systems).

Deriving a Standard — Three Steps

From one system-level control to a gated, catalogued standard

START · A SYSTEM-LEVEL CONTROL

e.g. "the system must resist prompt injection & jailbreaks" — EU AI Act Art 15

1 · Decompose

Break into testable best practices

- OWASP Top 10 LLM
- MITRE ATLAS
- NIST GenAI 600-1

not ISO 42001

2 · Crosswalk

Map each BP to the AWS GenAI Lens

- ✓ Covered → anchor
- ✕ Gap → new "na" standard

anchor at the implementation-step level

Lens = primary anchor

3 · Write

Make it real, provable & scoped

- Statement · Gate · Evidence
- Reference implementation
- Tag applies_to {ML · LLM · agent}

no gate, no standard

THE REUSABLE PATTERN

Control > Decompose > Crosswalk > Write

Covered by the Lens → anchor · not covered → a new standard beyond AWS (mapping_state "na")

Write = the standard (gate + evidence) + its reference implementation + the applies_to tag.

Step 1 · Decompose — Break the Control into Best Practices

Turn the legal "must" into testable, enforceable BP statements

INPUT **Art 15 — "the system must resist prompt injection & jailbreaks"**

SOURCE FRAMEWORKS

System-level technical frameworks the decomposition draws on

OWASP Top 10 for LLM Apps

The common LLM failure classes — prompt injection, insecure output handling, excessive agency.

MITRE ATLAS

Adversarial ML tactics & techniques — the "ATT&CK" catalogue for attacks on AI systems.

NIST GenAI Profile (AI 600-1)

Risk-management actions specific to generative AI, mapped to the NIST AI RMF functions.

Not ISO 42001.

That's the org-level management layer Governance already owns — using it here drags the work back up a level.

OUTPUT · SIX BEST PRACTICES

Each is a testable, enforceable control statement

- **B1** Instruction hierarchy enforced
- **B2** Sanitise untrusted input — incl. indirect / RAG injection
- **B3** Output constrained to a schema / allow-list
- **B4** Downstream tools run least-privilege
- **B5** Adversarial red-team passes before deploy
- **B6** Injection attempts detected & logged at runtime

One legal "must" becomes six things you can actually test.

Decompose from OWASP / MITRE ATLAS / NIST — the frameworks that describe how AI systems fail.

These BPs are the raw material for Step 2, where each is crosswalked to the AWS GenAI Lens.

Step 2 · Crosswalk — Map Each BP to the AWS GenAI Lens

Covered by the Lens → anchor · no analogue → a new standard

THE AWS GENAI LENS — STRUCTURE

Standards anchor at the implementation-step level

AWS GenAI Lens

- └ Pillar — **GENSEC** (Security)
  - └ Question — **GENSEC04** (Prompt Security)
    - └ Best Practice
      - └ Implementation Step — ◀ anchor here
        - └ **OUR STANDARD**

Each implementation step gets a due-diligence pass:

- **promote** → becomes a standalone standard
- **not_promote** → absorbed by a sibling / no distinct content
- **pending** → not yet reviewed

AWS defines the subject matter; our standard adds the WHERE / WHO / WHEN / HOW-you-prove-it.

THE DECISION — PER BEST PRACTICE

Every BP lands one of two ways

✓ **Covered by a Lens step → anchor to it**

✕ **No analogue → new standard, mapping_state "na"**

ART 15 · B1–B6 CROSSWALKED

B1 · B3 · B5 · B6 → GENSEC04 (anchor)

B4 → P21 · Identity & Access (anchor)

B2 (indirect / RAG injection) **→ GAP → new "na"**

The Lens covers direct user injection under Prompt Security; indirect injection via retrieved / tool content has no clean BP — so B2 becomes the new standard written in Step 3.

Covered BPs anchor to the Lens; the gap becomes a new standard.

The gap is where the catalogue earns its keep — it extends beyond AWS (mapping_state "na").

AWS would place many such gaps in its separate Responsible AI Lens.

Step 3 · Write — Standard, Reference Implementation & Tag

Make it real: a provable rule, a build others adopt, and a scope

① WRITE THE STANDARD

ST-GENSEC04-12

mapping_state: na

applies_to {LLM, agent}

STATEMENT

Retrieved & tool-returned content is treated as untrusted and sanitised before it enters the model context — it cannot carry instructions that alter system behaviour.

GATE (pass / fail)

Red-team suite injects adversarial payloads into the RAG corpus + tool responses; 0 instruction overrides allowed to pass the deploy gate.

EVIDENCE

Red-team report + runtime injection-detection logs (links B6).

No gate, no standard — just a wish.

Every standard must carry a pass/fail test and the evidence that proves it — this is the discipline that makes a standard implementable rather than aspirational.

Extends the Lens from direct to indirect injection.

② REFERENCE IMPLEMENTATION

The build others reuse — central vs project split

CENTRAL — Builds · Operates · Owns

Shared red-team harness + injection-detection service (B6)

PROJECT — Configures · Populates · Consumes

Registers its RAG corpus + tool set; runs the harness in CI

INTERFACE CONTRACT

Harness API + required pass threshold (0 overrides)

ACCEPTANCE CRITERIA

Deploy gate green only when the suite passes on the project's own corpus

Build once centrally; every project consumes the same gate.

③ TAG · applies_to

Scope the standard by system type

LLM | agent | ML ✕

A pure ML model has no prompt → excluded.

One catalogue, filtered by system type at build time.

Write = the rule + the reference implementation others adopt + the scope.

A standard isn't "done" without a gate, an RI teams can consume, and an applies_to tag.

PART 2 · BUILDING THE CATALOGUE

How a Candidate Becomes a Standard

1

Choose the Anchor

(which baselines we take — regulation, frameworks, or bespoke)

2

Keep or Drop

(GenAI-distinctive? already covered? enforceable? material?)

3

Score the Survivors

(criticality · difficulty · reusability · enforcement cost)

4

Write the Standard

(statement + gate + evidence + applies_to)